UNIVERSIDAD AMERICANA

Facultad de Ingeniería y Arquitectura

Programación orientada a objetos II - Grupo 3

❖ Primer Corte. Documentación de Evaluación integradora práctica (fundamentos Android + POO)

Elaborado por :

Kenneth Saúl Blandón Larios

Alfredo Sánchez Bojorge

Manuel Alejandro Jirón Martinez

Jonathan Josué Rivera Guido

Docente:

José Durán García

Fecha de entrega:

Miércoles 15 de abril 2026

Documentación del código: Aplicación de Registro de Asistencia

1. Descripción general

Este proyecto corresponde a una aplicación móvil desarrollada en Kotlin con Jetpack Compose y Material Design 3, cuyo propósito es permitir el registro diario de asistencia de estudiantes dentro del aula. La interfaz permite ingresar el nombre del estudiante, seleccionar su estado de asistencia y almacenar cada registro en una lista dinámica mostrada en pantalla.

La aplicación fue diseñada para ser sencilla, funcional y visualmente moderna. Además, incorpora manejo de estado con Compose, validación básica de datos y una presentación clara de la información mediante tarjetas de colores.

2. Objetivo de la aplicación

El objetivo principal de esta aplicación es facilitar el control de asistencia de estudiantes por medio de una interfaz intuitiva. El docente puede registrar rápidamente si un estudiante está **Presente**, **Ausente** o **Tarde**, y visualizar inmediatamente cada registro realizado dentro de una lista organizada.

3. Tecnologías utilizadas

El proyecto utiliza las siguientes tecnologías y herramientas:

Kotlin: lenguaje principal de programación.
Jetpack Compose: framework moderno para construir interfaces declarativas en Android.
Material Design 3: sistema de diseño utilizado para dar uniformidad, accesibilidad y apariencia moderna a la interfaz.
Android Studio: entorno de desarrollo utilizado para crear y ejecutar la aplicación.

4. Estructura general del código

El archivo contiene varios bloques importantes que trabajan en conjunto para construir la aplicación:

a) MainActivity

Es la actividad principal de la aplicación. Su función es iniciar la interfaz cuando la app se ejecuta. Dentro del método onCreate, se configura el contenido visual usando setContent y se aplica el tema RegistroAsistenciaTheme. Luego, se llama al composable principal ControlAsistenciaScreen, que es donde se construye toda la pantalla.

b) data class RegistroAsistencia

Esta clase de datos representa cada registro individual de asistencia. Contiene dos propiedades:

- nombre: almacena el nombre del estudiante.
- estado: guarda el estado de asistencia seleccionado.

Esta estructura permite organizar la información de forma clara y reutilizable.

c) enum class EstadoAsistencia

Es una enumeración que define los tres posibles estados de asistencia que puede tener un estudiante:

- PRESENTE
- AUSENTE
- TARDE

El uso de un enum mejora el control del programa, ya que evita errores al manejar estados como texto libre y garantiza que solo se utilicen valores válidos.

d) Funciones auxiliares del enum

El código incluye dos funciones privadas asociadas a EstadoAsistencia:

label()

Convierte cada valor interno del enum en un texto entendible para el usuario. Por ejemplo, PRESENTE se muestra como “Presente”. Esto mejora la presentación visual en la interfaz.

cardColor()

Asigna un color distinto a cada estado de asistencia:

- Verde claro para presente
- Rojo claro para ausente
- Amarillo claro para tarde

Esto permite identificar visualmente cada registro de manera rápida.

5. Funcionamiento de la pantalla principal

La función composable ControlAsistenciaScreen es el núcleo de la aplicación. Aquí se construye toda la interfaz visual y se maneja la lógica de interacción con el usuario.

Dentro de esta función se usan varios estados con remember:

- nombreEstudiante: guarda lo que el usuario escribe en el campo de texto.
- estadoSeleccionado: almacena el estado actual elegido por el usuario.
- mensajeConfirmacion: muestra mensajes dinámicos en pantalla.
- registros: lista mutable que almacena todos los registros de asistencia.

Esto permite que la interfaz reaccione automáticamente a los cambios, una de las principales ventajas de Jetpack Compose.

6. Componentes de la interfaz

- Título principal

En la parte superior se muestra el texto **“Control de Asistencia”**, que sirve como encabezado de la pantalla. Se usa una tipografía destacada con headlineSmall y FontWeight.Bold, lo cual le da jerarquía visual.

- Campo para nombre del estudiante

Se utiliza un OutlinedTextField para que el docente escriba el nombre del estudiante. Este campo ocupa todo el ancho disponible y está configurado en una sola línea para mantener la interfaz ordenada.

- Selector de estado

El estado de asistencia se elige mediante botones de opción (RadioButton). El programa recorre todos los valores del enum EstadoAsistencia.entries y crea una fila para cada uno. Esto permite que el usuario elija entre Presente, Ausente o Tarde de manera clara y sencilla.

- Botón para registrar asistencia

El botón Registrar asistencia activa la lógica principal del programa. Cuando se presiona, el sistema verifica primero si el nombre ingresado no está vacío. Si el campo está vacío, se muestra un mensaje de advertencia. Si el nombre es válido, se crea un nuevo objeto RegistroAsistencia, se agrega a la lista y se actualiza el mensaje dinámico de confirmación. Además, el campo de texto se limpia y el estado vuelve a “Presente” por defecto.

- Mensaje dinámico

Debajo del botón se muestra un mensaje dinámico que cambia según la acción del usuario. Puede indicar que se registró correctamente la asistencia o que se debe ingresar un nombre antes de continuar. Esto mejora la interacción y la retroalimentación inmediata al usuario.

- Lista de registros con LazyColumn

Los estudiantes registrados se muestran mediante LazyColumn, que es un componente optimizado para desplegar listas largas de manera eficiente. Cada nuevo registro se inserta al inicio de la lista, por lo que el más reciente aparece primero.

- Tarjetas de registro

Cada elemento de la lista se presenta en una Card. Dentro de ella se muestra:

- ❖ el número de orden del registro,
- ❖ el nombre del estudiante,
- ❖ el estado correspondiente.

El color de la tarjeta cambia según el estado de asistencia, lo cual hace que la información sea más visual y organizada.

Mensaje cuando no hay registros

Si la lista está vacía, la app muestra una tarjeta con el texto **“No hay estudiantes registrados todavía.”** Esto evita que la pantalla quede vacía y le comunica al usuario el estado actual del sistema.

7. Lógica de funcionamiento

La lógica principal del programa sigue esta secuencia:

Primero, el docente escribe el nombre del estudiante. Después, selecciona uno de los tres estados de asistencia disponibles. Luego, al presionar el botón de registro, el sistema limpia los espacios innecesarios del texto con trim() y valida si el nombre no está vacío. Si está vacío, la aplicación no registra nada y muestra un mensaje de advertencia. Si el nombre es correcto, crea un nuevo registro, lo agrega a la lista, actualiza el mensaje de confirmación y reinicia los valores del formulario.

Esta lógica garantiza un flujo simple, directo y funcional para el usuario.

8. Manejo de estado

Uno de los puntos más importantes del código es el uso de remember y mutableStateOf, ya que permiten conservar y actualizar la información de la interfaz durante la ejecución. También se utiliza mutableStateListOf para mantener una lista reactiva de registros. Cuando la lista cambia, Compose vuelve a dibujar automáticamente la interfaz con la información nueva.

Esto hace que la aplicación sea dinámica y que no sea necesario refrescar manualmente la pantalla.

9. Aspectos de diseño y accesibilidad

La aplicación sigue una estructura visual limpia y moderna gracias al uso de Material Design 3. Los textos son claros, los espacios entre componentes están bien definidos y los controles son fáciles de identificar. Además, el uso de colores diferenciados para las tarjetas facilita la lectura rápida del estado de cada estudiante.

En términos de accesibilidad, se puede decir que la app presenta etiquetas visibles, componentes grandes y comprensibles, y una organización adecuada de la información en pantalla.

10. Vista previa de la interfaz

La función GreetingPreview() permite visualizar la pantalla desde Android Studio sin necesidad de ejecutar la app en un dispositivo físico o emulador. Esto facilita la revisión visual del diseño durante el desarrollo.

13. Conclusión

En conclusión, el código implementa correctamente una aplicación de registro de asistencia escolar usando herramientas modernas de desarrollo Android. La interfaz cumple con los requerimientos funcionales del caso, permite interacción sencilla con el usuario, utiliza un manejo adecuado del

estado y presenta la información de forma ordenada mediante tarjetas y listas dinámicas. Por ello, se trata de una solución adecuada, clara y alineada con los principios de Material Design 3 y Jetpack Compose.